

Deep Learning

Contents

1	Word Embeddings	2
1.1	CBOW	2
1.2	Skip-gram	2
1.3	n-gram Models	2
1.4	Markov Models	2
1.5	TF-IDF	2
2	Perplexity	2
3	Neural Network Basics	3
3.1	Vanishing & Exploding Gradient Descent	3
3.2	Positional Embeddings	3
3.3	Covariate Shift and Normalization Methods	4
3.4	Byte Pair Encoding (BPE)	4
3.5	Dropout Mechanism	4
3.6	ReLu Activation	4
4	Sequential Architectures	4
4.1	RNN	5
4.2	LSTM	5
4.3	GRU	5
4.4	Encoder-Decoder with Attention	5
5	Transformers	6
5.1	GPT: Architecture and Training	7
5.2	BERT	7
5.3	BART	8
5.4	T5	8
5.5	Generative Adversarial Networks	9
6	BLEU Score (Bilingual Evaluation Understudy)	9
6.1	Modified n-gram Precision	9
6.2	Clipping	10
6.3	Brevity Penalty (BP)	10
7	BERTScore	10
7.1	BERT Precision	10
7.2	BERT Recall	11
8	ROUGE Score (Recall-Oriented Understudy for Gisting Evaluation)	11
8.1	ROUGE-N (n-gram Recall)	11
8.2	ROUGE Precision (optional)	11
8.3	ROUGE-L (Longest Common Subsequence)	12

1 Word Embeddings

Word embeddings are dense vectors that represent tokens in continuous space, capturing semantic and syntactic relationships.

- Fixed-size dense representation of words/tokens.
- Capture semantic closeness (similar words get nearby vectors).
- Examples: GloVe, FastText, ELMo, BERT embeddings.

1.1 CBOW

Continuous Bag of Words predicts the center word from surrounding context words.

$$P(w_t \mid w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c})$$

1.2 Skip-gram

Skip-gram predicts surrounding context words from a center word.

$$P(w_{t+j} \mid w_t), \quad j \in [-c, c], j \neq 0$$

1.3 n-gram Models

An n-gram model estimates the next-token probability from the previous $n - 1$ words.

$$P(w_t \mid w_{t-n+1}, \dots, w_{t-1})$$

For trigram:

$$P(w_i \mid w_{i-2}, w_{i-1})$$

Count-based estimation (as in your notebook) is:

$$P(w_i \mid w_{i-2}, w_{i-1}) = \frac{\#(w_{i-2}, w_{i-1}, w_i)}{\#(w_{i-2}, w_{i-1})}$$

1.4 Markov Models

Markov assumption: future depends only on a limited past (order k).

$$P(w_{n+1} \mid w_1, \dots, w_n) \approx P(w_{n+1} \mid w_{n-k+1}, \dots, w_n)$$

n-gram language models are a count-based realization of this assumption.

1.5 TF-IDF

TF-IDF scores term importance in a document collection.

$$\text{TF}(t, d) = \frac{\#(t \text{ in } d)}{\#(\text{terms in } d)}$$

$$\text{IDF}(t) = \log \frac{N}{1 + \text{DF}(t)}$$

$$\text{TFIDF}(t, d) = \text{TF}(t, d) \cdot \text{IDF}(t)$$

TF-IDF is useful as a lightweight text representation, but unlike neural embeddings, it does not directly model semantics/context.

2 Perplexity

Perplexity measures how uncertain a language model is on a sequence:

$$PP(W) = P(w_1, \dots, w_N)^{-1/N}$$

Normalized sequence probability viewpoint:

$$\text{Normalized Probability} = P(W)^{1/N}, \quad PP = \frac{1}{P(W)^{1/N}}$$

Equivalent log form:

$$PP(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid \text{history}) \right)$$

Equivalent product form:

$$PP(W) = \left(\prod_{i=1}^N \frac{1}{P(w_i \mid \text{history})} \right)^{1/N}$$

Lower perplexity indicates better predictive quality. Theoretical range: $PP \geq 1$ (best is 1; very poor models can have very large perplexity).

3 Neural Network Basics

3.1 Vanishing & Exploding Gradient Descent

Training deep neural networks involves computing gradients via repeated application of the chain rule. For a network with n layers, the gradient of the loss L with respect to parameters in early layers can be expressed as a product of intermediate derivatives:

$$\frac{\partial L}{\partial w} = \prod_{i=1}^n \frac{\partial a_i}{\partial a_{i-1}}.$$

This multiplicative structure leads to two well-known numerical pathologies.

If the magnitudes of the derivatives are typically less than 1, the product decays exponentially:

$$(0.5)^{10} = 0.000976,$$

resulting in vanishing gradients. In this regime, gradients become negligibly small, preventing effective learning in earlier layers and impairing the model's ability to capture long-range dependencies.

Conversely, if the derivatives are greater than 1, the product grows exponentially:

$$2^{10} = 1024,$$

leading to exploding gradients. This causes unstable parameter updates, numerical overflow, and potential divergence of the training process.

These issues are especially pronounced in deep feedforward networks and recurrent architectures, where long chains of multiplications are unavoidable.

Several remedies are commonly employed.

- Activation functions such as ReLU help maintain gradient magnitude by avoiding saturation.
- Proper weight initialization schemes (e.g., Xavier or He initialization) aim to preserve variance across layers, stabilizing both forward activations and backward gradients.
- Gradient clipping is an effective technique to control exploding gradients by rescaling them when their norm exceeds a predefined threshold:

$$g \leftarrow \frac{g}{\max(1, \|g\|/c)}.$$

- Normalization methods such as batch normalization reduce internal covariate shift and improve gradient flow.
- Architectural strategies, including residual or skip connections, provide shorter paths for gradient propagation and mitigate repeated multiplicative effects. In recurrent settings, gated units such as LSTM and GRU introduce mechanisms that regulate information flow and alleviate vanishing gradients.

3.2 Positional Embeddings

$$PE(pos, 2k) = \sin \left(\frac{pos}{10000^{2k/d}} \right), \quad PE(pos, 2k+1) = \cos \left(\frac{pos}{10000^{2k/d}} \right)$$

3.3 Covariate Shift and Normalization Methods

Covariate shift refers to changes in input feature distribution between training and deployment (or changing layer input distributions during optimization). Normalization methods stabilize optimization and speed up training:

- **Batch Normalization:** normalize per feature across the mini-batch.
- **Layer Normalization:** normalize across features within each sample.
- **Instance Normalization:** normalize each sample-channel independently (common in vision style tasks).
- **Group Normalization:** split channels into groups and normalize within each group.

3.4 Byte Pair Encoding (BPE)

BPE is a subword tokenization algorithm:

- Start with character-level symbols.
- Repeatedly merge the most frequent adjacent pair.
- Add merged pair to vocabulary and update corpus.
- Stop after a fixed number of merges.

It balances vocabulary size and out-of-vocabulary robustness.

3.5 Dropout Mechanism

Dropout randomly deactivates units during training with drop probability p . At inference, full network is used with scaling to keep expected activations consistent. Effect: regularization, reduced co-adaptation, better generalization.

- Random masking adds noise during training.
- Prevents neurons from becoming overly specialized or “lazy”.
- Encourages distributed responsibility across many units.

3.6 ReLU Activation

$$\text{ReLU: } f(x) = \max(0, x)$$

$$f'(x) = \begin{cases} 0 & x < 0 \\ 1 & x > 0 \end{cases}$$

ReLU passes positive activations unchanged and suppresses negative activations to zero, introducing non-linearity and sparsity in neural representations.

- **Vanishing gradient problem:** mitigated compared to sigmoid/tanh since $f'(x) = 1$ for $x > 0$, preventing repeated shrinkage of gradients across layers.
- **Exploding activations:** possible because outputs are unbounded for $x > 0$, leading to large forward and backward signal magnitudes in deep networks.
- **Dying ReLU problem:** if $z = Wx + b < 0$ for all inputs, then $f(z) = 0$ and $f'(z) = 0$, causing neurons to stop learning permanently.
- **Sparsity:** ReLU outputs exact zeros for negative inputs, producing sparse activations and efficient representations.
- **Unbounded positive outputs:** $f(x)$ grows linearly for $x > 0$, which can lead to unstable activations without normalization.
- **Instability across layers:** repeated application can cause variance shift if weights are not properly scaled.

For ReLU, approximately half activations are zero:

He initialization compensates this effect: $\text{Var}(W) = \frac{2}{n_{\text{in}}}$
ensuring stable variance across layers: $\text{Var}(z) \approx 1$

4 Sequential Architectures

4.1 RNN

Recurrent Neural Networks maintain hidden state over time:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \quad y_t = W_y h_t + c$$

Issue: vanishing/exploding gradients for long dependencies.

4.2 LSTM

LSTM introduces gated memory flow (forget, input, output gates):

$$\begin{aligned}\text{Forget Gate : } f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ \text{Input Gate : } i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\ \text{Candidate Memory : } \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\ \text{Cell State : } c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ \text{Output Gate : } o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\ \text{Hidden State : } h_t &= o_t \odot \tanh(c_t)\end{aligned}$$

4.3 GRU

GRU is a lighter gated recurrent unit with update and reset gates:

$$\begin{aligned}\text{Update Gate : } z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) \\ \text{Reset Gate : } r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) \\ \text{candidate hidden state : } \tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h) \\ \text{Hidden State : } h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t\end{aligned}$$

4.4 Encoder-Decoder with Attention

- In normal Encoder - Decoder models, Encoder reads input sequence and builds the context vector (the final hidden states). Decoder starts the generating output autoregressively using the previous outputs.(Decoder starts with the context vector)
- Instead of a single fixed context vector, attention creates a dynamic context at each decoder step.
- Encoder produces all hidden states $h_1, h_2, \dots, h_n$
- Decoder at each step t , looks at all encoder states and decides; which ones matter more right now.
- For each encoder state, we compute an alignment score against the previous decoder hidden state.
- Then we can normalize the score
- Then we can get the weighted sum of $h_1, h_2, \dots, h_n$. This is the context window. And this is dynamic , changes at every decoding step.

5 Transformers

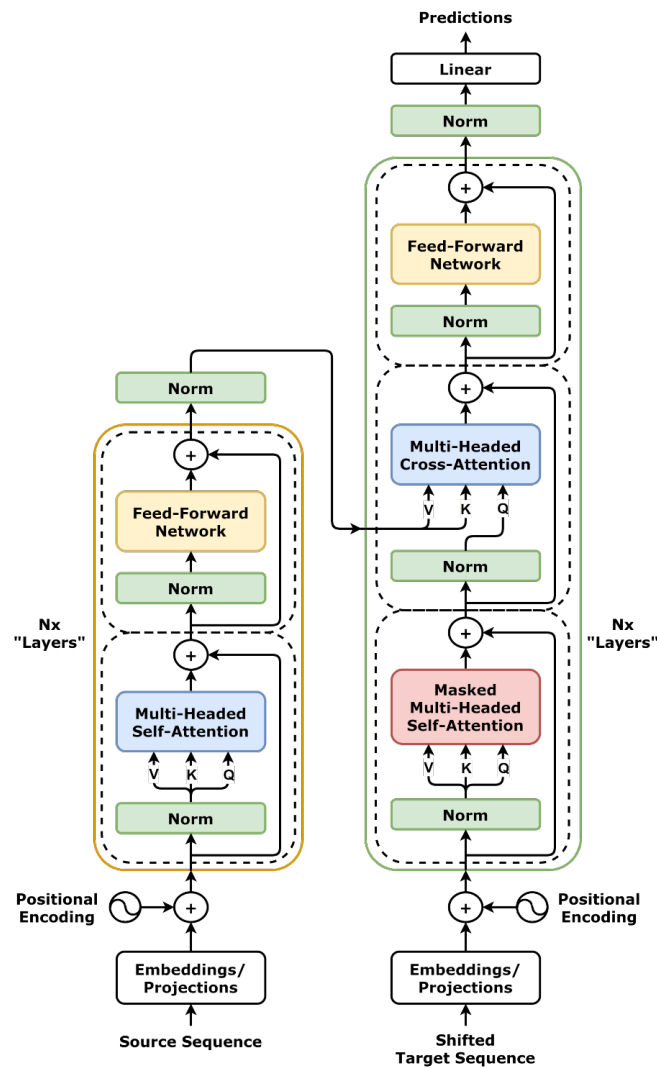


Figure 1: Transformer Architecture

Self-attention computes contextualized token representations by comparing Query, Key, and Value projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Each token becomes a weighted combination of value vectors from all tokens.

- Before attention, token embeddings are independent.
- After attention, each token mixes information from other tokens based on relevance weights.
- Intuition: compare “what I am looking for” (query) with “what each token offers” (key), then aggregate information (value).

Cross-attention is used in encoder-decoder models:

- Queries come from decoder states.
- Keys and values come from encoder outputs.
- At each decoding step, the decoder chooses which source tokens matter most.

Transformers are attention-based sequence models with strong parallelism.

- Input = token embeddings + positional embeddings.
- Encoder block: multi-head self-attention, feed-forward network, residual connections, layer norm.

- Decoder block: masked self-attention, cross-attention, feed-forward network, residual connections, layer norm.
- Output head: linear projection to vocabulary + softmax.

Residual structure:

$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Decoder masking enforces autoregressive next-token prediction.

Transformers are attention-based sequence models with strong parallelism.

- Input = token embeddings + positional embeddings.
- Encoder block: multi-head self-attention, feed-forward network, residual connections, layer norm.
- Decoder block: masked self-attention, cross-attention, feed-forward network, residual connections, layer norm.
- Output head: linear projection to vocabulary + softmax.

Residual structure:

$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Decoder masking enforces autoregressive next-token prediction.

5.1 GPT: Architecture and Training

GPT is a decoder-only transformer:

- Masked self-attention (causal mask) so token t attends only to $\leq t$.
- Trained with next-token prediction objective (cross-entropy).
- Inference is autoregressive: predict token, append, repeat.

Pipeline: tokenization $\rightarrow$ token + positional embeddings $\rightarrow$ stacked decoder blocks $\rightarrow$ linear head + softmax.

5.2 BERT

BERT is an encoder-only bidirectional transformer with input:

$$\text{Token Embedding} + \text{Position Embedding} + \text{Segment Embedding}$$

BERT input representation details:

- WordPiece token embeddings (about 30k vocabulary in the original setup).
- Segment embeddings distinguish sentence A vs sentence B.
- Position embeddings encode token order.
- Special tokens: [CLS] at start and [SEP] between/after sentences.

Pretraining objectives:

- **MLM**: predict masked tokens using both left and right context.
- **NSP**: classify whether sentence B follows sentence A.

BERT pretraining specifics from your notes:

- About 15% of tokens are selected for MLM.
- Of selected tokens: 80% replaced with [MASK], 10% random token, 10% unchanged.
- NSP sampling is typically 50% true next sentence, 50% random sentence.

BERT workflow has two stages:

- **Pretraining**: self-supervised learning on large unlabeled text.
- **Fine-tuning**: task-specific supervised training with a small task head.

BERT model sizes in your notebook:

- BERT-base: 12 layers, hidden size 768, 12 heads, around 110M parameters.
- BERT-large: 24 layers, hidden size 1024, 16 heads, around 340M parameters.

BERT is strong for understanding tasks: classification, NER, QA, and sentence-pair tasks.

- Usage tip: with absolute positional embeddings, right-padding is usually preferred.
- DistilBERT is a smaller variant with significant speedup and strong retained accuracy.
- BERT is excellent for NLU, but decoder models are better suited for free-form generation.

5.3 BART

BART (Bidirectional and Auto-Regressive Transformers) is a sequence-to-sequence model that combines a bidirectional encoder, similar to BERT, with an autoregressive decoder, similar to GPT. It is designed as a denoising autoencoder for pretraining sequence generation tasks, learning to reconstruct original text from corrupted input.

The architecture is based on the standard Transformer framework. The encoder consists of multiple stacked layers of self-attention and feedforward networks, where each token attends to all other tokens in the input sequence bidirectionally. Formally, given an input sequence $x = (x_1, x_2, \dots, x_n)$, the encoder produces contextual representations:

$$h = \text{Encoder}(x).$$

The decoder is also composed of stacked Transformer layers, but operates autoregressively. Each decoder layer includes masked self-attention, encoder-decoder cross-attention, and feedforward components. At time step t , the decoder generates a token conditioned on previously generated tokens and the encoder output:

$$p(y_t | y_{<t}, x) = \text{Decoder}(y_{<t}, h).$$

BART is pretrained using a denoising objective. The original input sequence x is corrupted using a stochastic transformation $C(x)$, and the model is trained to reconstruct the original sequence:

$$\mathcal{L} = - \sum_t \log p(x_t | x_{<t}, C(x)).$$

Common corruption strategies include token masking, token deletion, text infilling (replacing spans with a mask token), sentence permutation, and document rotation. These transformations force the model to learn both local and global dependencies in the data.

The encoder processes the corrupted input $C(x)$, producing latent representations:

$$h = \text{Encoder}(C(x)).$$

The decoder then generates the original sequence token by token using teacher forcing during training.

The self-attention mechanism in both encoder and decoder is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where Q, K, V are query, key, and value matrices derived from input embeddings.

The key design of BART lies in combining bidirectional context encoding with left-to-right generation, enabling strong performance across both understanding and generation tasks. After pretraining, the model can be fine-tuned for downstream tasks such as text summarization, machine translation, question answering, and dialogue generation by adapting the decoder output to task-specific formats.

5.4 T5

T5 (Text-to-Text Transfer Transformer) is a unified sequence-to-sequence Transformer architecture that reformulates all natural language processing tasks into a text-to-text format. Both inputs and outputs are treated as text strings, enabling a single model to handle diverse tasks such as translation, summarization, classification, and question answering within the same framework.

The architecture follows an encoder-decoder Transformer design. Given an input sequence $x = (x_1, x_2, \dots, x_n)$, the encoder produces contextual representations:

$$h = \text{Encoder}(x).$$

The decoder generates the output sequence $y = (y_1, y_2, \dots, y_m)$ autoregressively:

$$p(y_t | y_{<t}, x) = \text{Decoder}(y_{<t}, h).$$

Each layer in both encoder and decoder consists of multi-head attention and position-wise feedforward networks. The attention mechanism is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

A distinguishing feature of T5 is its use of a text-to-text paradigm. Every task is converted into a textual input-output pair using task-specific prefixes. For example:

"translate English to German: That is good" → "Das ist gut".

T5 is pretrained using a span-corruption objective. Given an input sequence, contiguous spans of tokens are replaced with unique sentinel tokens, and the model is trained to predict the missing spans in order. Let $C(x)$ denote the corrupted sequence; the objective is:

$$\mathcal{L} = - \sum_t \log p(x_t \mid x_{<t}, C(x)).$$

Unlike standard masking strategies, the targets consist only of the removed spans, each preceded by its corresponding sentinel token. This allows the model to learn structured reconstruction and long-range dependencies.

T5 uses relative position embeddings instead of absolute positional encodings, improving generalization to longer sequences. The model is trained on a large-scale dataset (C4: Colossal Clean Crawled Corpus), emphasizing transfer learning across tasks.

During fine-tuning, no architectural changes are required. Tasks are specified purely through input text formatting, and the model generates outputs accordingly. This unified formulation simplifies the pipeline and enables strong performance across a wide range of NLP benchmarks.

5.5 Generative Adversarial Networks

Generative Adversarial Networks (GANs) are a class of generative models that learn to approximate a target data distribution by framing the learning problem as a two-player minimax game. The framework consists of two neural networks: a generator G and a discriminator D , both parameterized by differentiable functions.

The generator $G(z)$ maps a latent variable $z \sim p_z(z)$, typically drawn from a simple prior such as a standard normal distribution, into the data space. Its objective is to produce samples that resemble the true data distribution $p_{\text{data}}(x)$. The discriminator $D(x)$ outputs a scalar in $[0, 1]$, representing the probability that input x is drawn from the real data rather than generated.

The interaction between these two networks is formalized as a minimax optimization problem:

$$\min_G \max_D V(D, G)$$

where the value function is given by

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))].$$

The discriminator seeks to maximize this objective by correctly classifying real samples as real and generated samples as fake. Conversely, the generator seeks to minimize the same objective by producing samples that maximize the discriminator's error.

6 BLEU Score (Bilingual Evaluation Understudy)

BLEU is a corpus-level metric for evaluating generated text by comparing it with one or more reference texts. It is primarily based on **modified n-gram precision** with a **brevity penalty** to discourage overly short outputs.

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

- p_n : modified precision for n-grams
- w_n : weights (typically uniform, e.g., $w_n = \frac{1}{N}$)
- BP : brevity penalty

6.1 Modified n-gram Precision

$$p_n = \frac{\sum_{\text{n-grams}} \min(\text{count}_{\text{cand}}, \text{count}_{\text{ref}})}{\sum_{\text{n-grams}} \text{count}_{\text{cand}}}$$

Intuition:

- Measures: “How many generated n-grams are correct?”
- Denominator = total candidate n-grams
- Numerator = matching n-grams (with clipping)

6.2 Clipping

Each n-gram match is **capped** by its maximum frequency in the reference:

$$\text{count}_{\text{clip}} = \min(\text{count}_{\text{cand}}, \text{count}_{\text{ref}})$$

This prevents artificially inflating precision by repetition.

If a sentence has N tokens, the number of n-grams is:

$$N - n + 1$$

- Unigram ($n = 1$): N
- Bigram ($n = 2$): $N - 1$
- Higher n : stricter matching

6.3 Brevity Penalty (BP)

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

- c : candidate length
- r : reference length

Behavior:

- If $c < r$: $BP < 1$ (penalty applied)
- As $c \ll r$: $BP \rightarrow 0$ (strong penalty)
- If $c > r$: $BP = 1$ (no penalty)
- Precision-based (no recall)
- Uses geometric mean across n-grams
- Penalizes short outputs via BP
- Sensitive to exact wording (not semantics)

7 BERTScore

BERTScore is a metric for evaluating generated text using **contextual embeddings** from transformer models (e.g., BERT). Instead of exact n-gram overlap, it measures **semantic similarity** between candidate and reference tokens. Each token in the candidate and reference is mapped to a contextual embedding. Similarity between tokens is computed using cosine similarity, allowing matching of **paraphrases and semantically similar words**.

Let:

- $\mathbf{x}_i$: embedding of i -th candidate token
- $\mathbf{y}_j$: embedding of j -th reference token

$$\text{sim}(\mathbf{x}_i, \mathbf{y}_j) = \frac{\mathbf{x}_i \cdot \mathbf{y}_j}{\|\mathbf{x}_i\| \|\mathbf{y}_j\|}$$

7.1 BERT Precision

$$P_{\text{BERT}} = \frac{1}{|x|} \sum_{i=1}^{|x|} \max_j \text{sim}(\mathbf{x}_i, \mathbf{y}_j)$$

Intuition:

- For each candidate token, find the **most similar** token in the reference
- Average these similarities

Interpretation:

- “How much of the generated text is semantically correct?”

7.2 BERT Recall

$$R_{\text{BERT}} = \frac{1}{|y|} \sum_{j=1}^{|y|} \max_i \text{sim}(\mathbf{x}_i, \mathbf{y}_j)$$

Intuition:

- For each reference token, find the **best matching** token in the candidate
- Average these similarities

Interpretation:

- “How much of the reference meaning is captured by the generated text?”
- Captures **semantic similarity**, not exact matches
- Handles **paraphrases and synonyms**
- Token-level alignment via **maximum similarity matching**
- Uses contextual embeddings (context-dependent meaning)
- BLEU: surface-level n-gram precision
- BERTScore: embedding-based semantic similarity
- BLEU fails on paraphrases; BERTScore handles them better

8 ROUGE Score (Recall-Oriented Understudy for Gisting Evaluation)

ROUGE is a set of metrics used to evaluate generated text (e.g., summaries) by comparing it with reference text(s). Unlike BLEU, ROUGE is primarily **recall-oriented**, focusing on how much of the reference content is captured by the candidate.

ROUGE measures **overlap between candidate and reference**, typically using:

- n-grams (ROUGE-N)
- longest common subsequence (ROUGE-L)

8.1 ROUGE-N (n-gram Recall)

$$\text{ROUGE-N} = \frac{\sum_{\text{n-grams}} \min(\text{count}_{\text{cand}}, \text{count}_{\text{ref}})}{\sum_{\text{n-grams}} \text{count}_{\text{ref}}}$$

Intuition:

- Measures: “How much of the reference n-grams are recovered by the candidate?”
- Numerator = overlapping n-grams (with clipping)
- Denominator = total n-grams in the reference

8.2 ROUGE Precision (optional)

$$P_{\text{ROUGE-N}} = \frac{\sum_{\text{n-grams}} \min(\text{count}_{\text{cand}}, \text{count}_{\text{ref}})}{\sum_{\text{n-grams}} \text{count}_{\text{cand}}}$$

Interpretation:

- “How much of the generated content is relevant?”

If a sequence has N tokens, the number of n-grams is:

$$N - n + 1$$

8.3 ROUGE-L (Longest Common Subsequence)

ROUGE-L is based on the **Longest Common Subsequence (LCS)**, which captures sentence-level structure without requiring consecutive matches.

$$R_{\text{LCS}} = \frac{\text{LCS}(x, y)}{|y|} \quad P_{\text{LCS}} = \frac{\text{LCS}(x, y)}{|x|}$$

$$F1_{\text{LCS}} = \frac{2 \cdot P_{\text{LCS}} \cdot R_{\text{LCS}}}{P_{\text{LCS}} + R_{\text{LCS}}}$$

Intuition:

- Captures in-order matching (not necessarily contiguous)
- Rewards fluency and structure better than pure n-grams